

A General Randomness Reuse Framework for First-Order Secure Masking and Applications on AES Design Using Randomness Solely for Input Encoding

Junhuai Yang^{1,2}, Feng Zhou^{1,3}, Hua Chen^{1,3}, Limin Fan³

¹ TCA Laboratory, Institute of Software, CAS, Beijing, China

² University of Chinese Academy of Sciences, Beijing, China

³ Zhongguancun Laboratory

Abstract. Masking is a principal countermeasure against side-channel attacks, yet its practical application is often hindered by its high randomness cost. While randomness reuse is a common efficiency strategy, securely managing the resulting algebraic dependencies without significant overhead has been a central challenge. Previous approaches have either resorted to injecting costly extra randomness to ensure compositional security in hardware, or remained confined to theoretical proposals with impractical latency costs. Critically, a systematic methodology to resolve randomness dependencies and achieve provable security at a practical cost has been lacking. This paper addresses the challenge by introducing the Unique Randomness Guard (URG) property as a foundational principle for secure masked circuit design. Building upon this property, we formally establish conditions under which the randomness in a circuit remains uniform and independent throughout its computation. Based on these results, we construct a general framework that provides a systematic methodology for randomness reuse, defined through a set of formally verified rules for both parallel and serial compositions. The security of these rules is guaranteed by the URG-based theoretical results, ensuring constructions that are first-order secure under the robust probing model. We also present a first-order masked implementation of the AES. To the best of our knowledge, this hardware implementation is the first to be proven first-order secure in the robust probing model while using only the randomness required for input encoding, all while maintaining a competitive hardware area. This work ultimately validate that randomness reuse can be fully realized without compromising rigorous verifiability, establishing a new framework that directly addresses the central challenge of randomness cost in masked implementations.

Keywords: Hardware Masking · Randomness Reuse · AES · Provable Security

1 Introduction

As Internet of Things (IoT) devices proliferate across domains from smart homes to critical infrastructure, the security of their embedded cryptographic applications has become a cornerstone of digital trust. Classical cryptography, however, often assumes an idealized “black-box” model where algorithms are only vulnerable to mathematical attacks. In practice, this assumption is fundamentally challenged by Side-Channel Attacks (SCAs), which exploit physical leakages such as power consumption, electromagnetic emissions, and timing information to extract secret keys directly from hardware implementations [KJJ99, Koc96].

Among the various countermeasures, masking has emerged as a leading provably secure technique. Based on secret sharing, it splits sensitive variables into multiple shares such that any subset below a threshold reveals no information about the secret [Mes01, GP99, CJRR99]. The

security of such schemes was formalized by Ishai, Sahai, and Wagner in the probing model, which defines an implementation as d -th order secure if any combination of up to d intermediate values is statistically independent of the secret [ISW03]. While this model provides a rigorous foundation, practical hardware effects—notably glitches—introduce additional leakage [NRR06], motivating stronger frameworks such as the robust probing model [FGM⁺18].

A central milestone in this context is Threshold Implementation (TI), which enforces correctness, non-completeness, and uniformity to provide provable resistance against glitches [NRR06]. However, these guarantees often come with substantial cost: the original construction required $td + 1$ shares to protect a function of algebraic degree t at security order d [DNR19]. For complex nonlinear functions such as the AES S-box ($t = 7$), this leads to prohibitive area overhead. The standard solution is to decompose complex functions into quadratic gadgets [MPL⁺11, BGN⁺15, GMK17, DRB⁺16, UHA17, UHA17, WM18]. This strategy, while managing the share count, amplifies the need for fresh randomness, as the uniformity property—a prerequisite for secure composition—must be restored after each non-linear stage.

In parallel, a separate lineage of masking schemes [RP10, CPRR14, RBN⁺15, DRB⁺16, GMK17], evolving from [ISW03] achieved higher share-count efficiency, culminating in constructions requiring only the theoretical minimum of $d + 1$ shares. The trade-off for their share-count efficiency, however, is a significant reliance on fresh randomness. Specifically, properties like uniformity are not inherently preserved across nonlinear operations and must be restored by consuming new random values.

Thus, while starting from different principles, both major masking lineages converge on a common bottleneck in their practical application: a substantial dependence on fresh, online randomness. Approaches to reduce the randomness footprint of masked implementations can be broadly categorized into two synergistic philosophies: optimizing the internal design of cryptographic gadgets and managing randomness at the architectural level across multiple components.

To reduce the randomness footprint, one approach focuses on the meticulous, component-level construction of the masked primitives, aiming to make them inherently efficient and negate the need for randomness [WM18, Sug18, SM21, BDRS22]. Alternatively, an architectural-level strategy reduces randomness costs by reusing it across the entire system. The most prominent technique in this category is Changing of the Guards (COTG), which substitutes fresh randomness with shares of other, unrelated state variables—referred to as guards [CPRR14]. The methodology of randomness reuse has evolved, expanding from techniques tailored to functions with specific algebraic properties, such as bijections, to more systematic strategies applicable to a wider range of complex cryptographic structures [GKM⁺24]. The selection of values for reuse, such as the “guards” must be carefully managed in accordance with the algorithm’s internal structure and diffusion properties to prevent introducing new vulnerabilities. Indeed, this careful management is necessitated by the complex dependencies that such reuse schemes introduce—a characteristic that, while enabling high efficiency, also poses a significant challenge for formal verification. Consequently, theoretical security guarantees are often restricted to a single round or a “super-box” level. Extending these proofs to achieve provable multi-round security generally requires the periodic injection of fresh randomness [WM18, Sug18, GKM⁺24, YCW⁺24], or relies on some extra initial randomness excluding randomness for input encoding as this is the primary mechanism to break the long-term dependencies that otherwise make the security analysis intractable.

Existing researches have usually combined the two philosophies, integrating fine-grained component optimizations with global reuse strategies. While such hybrid approaches achieve notable efficiency gains, their applicability is often constrained by the structural details of the underlying components [YCW⁺24, WM18, Sug18, GKM⁺24]. Ultimately, their security guarantee over multiple rounds still relies on the availability of either initial or online randomness. However, in practical hardware implementations, generating high-quality randomness—whether through initial injection or an online Random Number Generator (RNG)—typically entails

significant overhead in terms of area and power consumption. This is especially pronounced for resource-constrained devices like IoT nodes and smart cards. Therefore, ensuring the security of cryptographic implementations under severely limited randomness resources has become a critical problem to be solved.

Contributions

In this work, we make three main contributions:

1. **A new theoretical perspective on the role of randomness in masking schemes.** We introduce a new formal language to formally model the dependency of probing security on the underlying randomness, providing a sound basis for analyzing and solving randomness dependencies.
2. **A new framework for randomness reuse.** Building on this theoretical insight, we propose a new and systematic algorithmic framework for randomness reuse. This framework provides the first formal method for solving randomness dependencies that naturally arise in reuse schemes. It is not tied to fine-grained component-level optimization and demonstrates how randomness can be securely reused without additional randomness being injected during operation, thereby resolving the randomness bottleneck in a principled manner.
3. **A low-randomness AES implementation with formal security proof.** We apply our algorithm to a full AES implementation and formally prove its security. The resulting construction achieves first-order masking security against glitches while using no additional randomness beyond that required for masking the plaintext and key. This makes our work the first provably secure masked implementation of a complete block cipher that eliminates the need for an online random number generator (RNG), offering significant practical advantages for area-constrained devices where RNGs are costly components.

2 Preliminaries

This section lays the theoretical groundwork for our security analysis. We first formalize computations as circuits and fix the notation used throughout the paper. We then describe how unmasked circuits are systematically transformed into masked implementations through a *circuit compiler*. Finally, we introduce probing models that capture adversarial capabilities in both idealized and physically-aware settings, culminating in an information-theoretic security criterion that serves as our formal target.

2.1 Circuit Model and Notation

We employ a computational model adapted from [ISW03], where a computation is formalized as a circuit C . Specifically, a circuit in our context consists of the following components:

- **Inputs and outputs.** Inputs are collected in the set of random variables $\mathcal{X}$, and outputs in $\mathcal{Y}$. We assume all inputs are secret.
- **Gates.** Elementary Boolean operators (AND, XOR, NOT) that form combinational logic.
- **Wires.** Directed edges that carry values between gates, collected in the set $\mathcal{W}$. At runtime, each wire $w \in \mathcal{W}$ carries a random variable.
- **Registers.** Sequential elements that separate combinational blocks in time.

Formally, C is represented as a directed acyclic graph (DAG) at the level of combinational blocks separated by registers. Notational conventions: lowercase letters (x, y, a, b, q, r) denote random variables on wires; calligraphic letters denote sets, e.g., $\mathcal{X}$ for inputs, $\mathcal{R}$ for randomness sources, $\mathcal{W}$ for all wires, $\mathcal{Q} \subseteq \mathcal{W}$ for candidate probing points, and $\mathcal{Y}$ for outputs.

2.2 From Standard Circuits to Masked Implementations

The core countermeasure replaces raw computations by their masked equivalents. A *circuit compiler* CC transforms a standard circuit C into a masked circuit $\hat{C}$, in which wires carry shared values instead of raw secrets. Early works on higher-order masking and secure compilation techniques include the seminal paper by Coron et al. (2000) introducing d -th order Boolean masking for side-channel protection, and Chari et al. (1999) presenting foundational definitions for probing security against side-channel attacks.

Definition 1 (Circuit Compiler). A circuit compiler consists of three algorithms:

Encoding (Enc) Randomly splits each input $x \in \mathcal{X}$ into $d+1$ Boolean shares $(x_0, \dots, x_d)$ such that $x = \bigoplus_{j=0}^d x_j$. Typically, to protect against d -th order side-channel attacks, inputs are encoded as $(d+1)$ -order Boolean masks. The share vector of x is denoted $\mathbf{x} = (x_0, \dots, x_d)$.

Compilation (CC) Maps C to $\hat{C} = \text{CC}(C)$ by replacing each gate with an appropriate share-based gadget.

Decoding (Dec) Recombines output shares into the original outputs.

Correctness. The compiler preserves semantics:

$$\text{Dec}(\text{CC}(C)(\text{Enc}(\mathcal{X}^*))) = C(\mathcal{X}^*), \quad \forall \mathcal{X}^* \in \mathbb{F}_2^{|\mathcal{X}|},$$

where $*$ indicates a particular assignment.

2.3 Probing Models for Security Analysis

As outlined in the introduction, probing models serve as the standard formalism for evaluating the resistance of masked circuits to side-channel attacks. This section recalls their formal definitions, following a notation similar to that used in [DBR19], which will be employed in the subsequent analysis.

In the d -probing model, the adversary may simultaneously observe up to d variables from $\mathcal{W}$, the set of all wire-associated random variables of the masked circuit $\hat{C}$. A single probe corresponds to some $q \in \mathcal{W}$, and a set of d probes corresponds to $\mathcal{P} = \{q_1, \dots, q_d\} \subseteq \mathcal{W}$.

Definition 2 (d -Probing Security [ISW03]). A masked circuit $\hat{C}$ is d -probing secure if, for any $\mathcal{P} \subseteq \mathcal{W}$ with $|\mathcal{P}| = d$, the joint distribution of the probed variables is statistically independent of any sensitive input $\mathcal{X}^* \in \text{Dom}(\mathcal{X})$. Formally,

$$\Pr(\mathcal{P} \mid \mathcal{X}^*) = \Pr(\mathcal{P}),$$

which ensures that no information about $\mathcal{X}^*$ can be inferred from the observation of at most d wires.

Although this model provides a clean abstraction, it neglects hardware-level effects such as glitches, i.e., transient switching behaviors that may cause a wire's instantaneous value to depend on all combinational inputs. To account for this, the robust probing model extends the semantics of a probe: instead of observing only one variable, a probe on a wire includes both the output and all its driving inputs. Each probe is therefore associated with a subset $\mathcal{Q} \subseteq \mathcal{W}$ determined by the structure of the circuit—for instance, a combinational gate induces a set consisting of its output together with all its input variables, while registers or latches delimit such sets in sequential logic. Probing d wires then reveals the union $\mathcal{P}_{\text{robust}} = \bigcup_{i=1}^d \mathcal{Q}_i$.

Definition 3 (Robust d -Probing Security [FGM⁺18]). A masked circuit is robust d -probing secure if, for any collection of d extended probes, the joint distribution of all revealed variables is statistically independent of the sensitive input $\mathcal{X}^*$. That is,

$$\Pr(\mathcal{P}_{\text{robust}} \mid \mathcal{X}^*) = \Pr(\mathcal{P}_{\text{robust}}).$$

It is immediate that robust d -probing security implies standard d -probing security, as each extended probe covers at least one wire variable. The robust model therefore enforces a strictly stronger requirement, while more faithfully capturing the leakage behavior observed in practice.

3 Some Theoretical Foundations

Formal frameworks for probing security, such as those for Threshold Implementations (TI), have established foundational properties like non-completeness and uniformity. However, these properties are typically described at the component level. This abstraction presents a challenge during composition, as it becomes difficult to systematically track how random variables propagate between components. To address this limitation, this section introduces a new formalism. Our approach enables a more direct analysis of the interplay between randomness and security, providing a clearer path to rigorous security proofs and facilitating the design of countermeasures with lower randomness overhead.

Notation. Let the original inputs of the circuit C be denoted by $\mathcal{X}$, and let the set of random variables be

$$\mathcal{R} = \mathcal{R}_{\text{in}} \cup \mathcal{R}_{\text{int}},$$

$\mathcal{R}_{\text{in}}$ denotes randomness consumed by the encoder to mask the inputs, and $\mathcal{R}_{\text{int}}$ denotes fresh randomness introduced inside the masked circuit $\hat{C}$. Let $\mathcal{A} = \{a^1, \dots, a^n\}$ denote the internal variables (signals) of the masked circuit.

We use the symbol F to denote a Boolean function

$$F : \mathbb{F}_2^{|\mathcal{S}|} \rightarrow \mathbb{F}_2,$$

where $\mathcal{S}$ is a source random variable vector. When applied to a concrete assignment of $\mathcal{S}$, denoted by $\mathcal{S}^*$, F returns the corresponding Boolean value. When the subscript $\mathcal{S}$ is attached, i.e. $F_{\mathcal{S}}$, it denotes the random variable obtained by evaluating F on the random vector $\mathcal{S}$.

In particular, each $a \in \mathcal{A}$ can be written as

$$a = F_{\mathcal{X} \cup \mathcal{R}},$$

meaning that a is the random variable generated by some Boolean function of the coordinates in $\mathcal{X} \cup \mathcal{R}$.

Definition 4 (Unique Randomness Guard (URG)). Let $m := |\mathcal{R}_2|$ and $\mathcal{R}_2 \subseteq \mathbb{F}_2^m$. Suppose there exists a function

$$H : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^n$$

such that each element $a^i \in \mathcal{A}$ can be expressed as

$$a^i = F_{\mathcal{X} \cup (\mathcal{R} \setminus \mathcal{R}_2)}^i \oplus H_{\mathcal{R}_2}^i,$$

where $H^i(\cdot)$ denotes the i -th coordinate function of H .

We say that $\mathcal{A}$ possesses a set of unique randomness guards or $\mathcal{A}$ holds the URG property if H is balanced, we denote this by $H \sim \mathcal{U}(\mathbb{F}_2^n)$. That is, for all $y \in \mathbb{F}_2^n$, the cardinality of the preimage $|H^{-1}(y)|$ is constant (specifically 2^{m-n}).

Theorem 1 (URG Implies Mutual Independence). *Let $\mathcal{A} = \{a^1, \dots, a^n\}$ possess the URG property with guard randomness $\mathcal{R}'$. Then, the variables $a^1, \dots, a^n$ are mutually independent, or jointly uniform.*

Proof. Let $m := |\mathcal{R}'|$ and let $\Lambda := \mathcal{X} \cup (\mathcal{R} \setminus \mathcal{R}')$ denote the set of non-guard variables. Fix an arbitrary assignment Λ^* . This determines a constant vector $\mathcal{C}^* \in \mathbb{F}_2^n$ defined by $\mathcal{C}^* = \mathbf{F}_{\Lambda^*}$.

For any specific assignment $\mathcal{A}^* \in \mathbb{F}_2^n$, the conditional probability is derived via the counting measure on the randomness space $\mathbb{F}_2^m$:

$$\begin{aligned} \Pr[\mathcal{A} = \mathcal{A}^* \mid \Lambda = \Lambda^*] &= \frac{1}{2^m} \cdot |\{\mathcal{R}'^* \in \mathbb{F}_2^m \mid \mathcal{C}^* \oplus \mathbf{H}(\mathcal{R}'^*) = \mathcal{A}^*\}| \\ &= \frac{1}{2^m} \cdot |\{\mathcal{R}'^* \in \mathbb{F}_2^m \mid \mathbf{H}(\mathcal{R}'^*) = \mathcal{A}^* \oplus \mathcal{C}^*\}| \\ &= \frac{1}{2^m} \cdot |\mathbf{H}^{-1}(\mathcal{A}^* \oplus \mathcal{C}^*)|. \end{aligned}$$

By the URG definition, $\mathbf{H}$ is balanced, implying $|\mathbf{H}^{-1}(\mathcal{V}^*)| = 2^{m-n}$ for any vector $\mathcal{V}^* \in \mathbb{F}_2^n$. Substituting this into the equation:

$$\Pr[\mathcal{A} = \mathcal{A}^* \mid \Lambda = \Lambda^*] = \frac{2^{m-n}}{2^m} = 2^{-n}.$$

Since this probability is constant for all $\mathcal{A}^*$, the joint distribution factorizes into the product of marginal probabilities:

$$\Pr[\mathcal{A} = \mathcal{A}^*] = 2^{-n} = \prod_{i=1}^n \frac{1}{2} = \prod_{i=1}^n \Pr[a^i = \mathcal{A}_i^*],$$

where $\mathcal{A}_i^*$ denotes the i -th coordinate of the assignment $\mathcal{A}^*$. Thus, the variables $a^1, \dots, a^n$ are mutually independent. $\square$

Consequently, the URG property is strictly stronger than the general notion of uniformity. While uniformity typically requires that each variable be uniformly distributed individually, it does not guarantee the mutual independence between variables—a property that is inherent to URG. This implies that within a set of variables possessing the URG property, any subset can serve as the randomness for the complementary subset.

Theorem 2 (Preservation of URG Property under Surjective Linear Mappings). *Let the set of variables $\mathcal{A}$ possess a URG of the form:*

$$\mathcal{A} = \mathbf{F}_{\mathcal{X} \cup (\mathcal{R} \setminus \mathcal{R}')} \oplus \mathbf{H}_{\mathcal{R}'}$$

Suppose the map $\mathbf{G} : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^k$ is both linear and surjective. Then the new set of variables $\mathcal{A}' = \mathbf{G}_{\mathcal{A}}$ holds the URG property.

Proof. Since the function $\mathbf{G}$ is linear, its application distributes over the XOR sum. This allows the new variable set $\mathcal{A}'$ to be expressed using the composed functions:

$$\mathcal{A}' = \mathbf{G} \circ (\mathbf{F}_{\mathcal{X} \cup (\mathcal{R} \setminus \mathcal{R}')} \oplus \mathbf{H}_{\mathcal{R}'}) = (\mathbf{G} \circ \mathbf{F})_{\mathcal{X} \cup (\mathcal{R} \setminus \mathcal{R}')} \oplus (\mathbf{G} \circ \mathbf{H})_{\mathcal{R}'}$$

Let $\mathbf{F}' := \mathbf{G} \circ \mathbf{F}$ and $\mathbf{H}' := \mathbf{G} \circ \mathbf{H}$. To show that $\mathcal{A}'$ satisfies the URG property, it suffices to prove that the composed guard function $\mathbf{H}' : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^k$ is balanced.

The function $\mathbf{H}$ is balanced by the definition of URG for $\mathcal{A}$. This implies that for any specific value $\mathcal{Y}^* \in \mathbb{F}_2^k$, the preimage size is constant:

$$|\mathbf{H}^{-1}(\mathcal{Y}^*)| = 2^{m-n}.$$

Similarly, since G is a surjective linear map, it is inherently balanced. For any target value $\mathcal{Z}^* \in \mathbb{F}_2^k$, the set of its preimages under G has a constant size determined by the kernel dimension:

$$|G^{-1}(\mathcal{Z}^*)| = 2^{n-k}.$$

Now, consider the preimage size of the composite function H' for an arbitrary output $\mathcal{Z}^* \in \mathbb{F}_2^k$:

$$\begin{aligned} |H'^{-1}(\mathcal{Z}^*)| &= \sum_{\mathcal{Y}^* \in G^{-1}(\mathcal{Z}^*)} |H^{-1}(\mathcal{Y}^*)| \\ &= \sum_{\mathcal{Y}^* \in G^{-1}(\mathcal{Z}^*)} 2^{m-n} \\ &= |G^{-1}(\mathcal{Z}^*)| \cdot 2^{m-n} \\ &= 2^{n-k} \cdot 2^{m-n} = 2^{m-k}. \end{aligned}$$

Since the preimage size is constant for all $\mathcal{Z}^*$, the function H' is balanced. Therefore, $\mathcal{A}'$ holds the URG property. $\square$

Next, we establish the main consequence of a set of variables possessing the URG property: any variable derived from such a set remains independent of the secret.

Theorem 3 (Independence under Boolean Functions). *Let $\sigma(\mathcal{A})$ denote the set of all variables generated by Boolean functions over $\mathcal{A}$. If $\mathcal{P} \subseteq \sigma(\mathcal{A})$, then*

$$\Pr[\mathcal{P} \mid \mathcal{X}] = \Pr[\mathcal{P}],$$

i.e., $\mathcal{P}$ is independent of $\mathcal{X}$.

Proof. From the IRG and URG theorems, $\mathcal{A}$ is uniform and independent of $\mathcal{X}$. Let $\mathcal{P} = (p^1, \dots, p^k)$ where $p^j = G^j(\mathcal{A})$ for Boolean functions G^j . Then

$$\begin{aligned} \Pr[p^k = p^{k*} \mid \mathcal{X} = \mathcal{X}^*] &= \sum_{\mathcal{A}^* : G^k(\mathcal{A}^*) = p^{k*}} \Pr[\mathcal{A} = \mathcal{A}^* \mid \mathcal{X} = \mathcal{X}^*] \\ &= \sum_{\mathcal{A}^* : G^k(\mathcal{A}^*) = p^{k*}} \Pr[\mathcal{A} = \mathcal{A}^*] \\ &= \Pr[p^k = p^{k*}]. \end{aligned}$$

Thus $\mathcal{P}$ is independent of $\mathcal{X}$. $\square$

URG variables provide a robust foundation for secure masked computations. Any variable derived from URG variables via boolean functions also remains independent of $\mathcal{X}$. The core principle of these definitions is as follows: for a given probing set, if a unique set of randomness guards can be established for its generating set, then the probing set is guaranteed to be independent of any secret variable. In the context of the robust probing model, the inputs of a combinational circuit naturally serve as such a generating set. Therefore, as long as we can find a set of unique randomness guards for the circuit's inputs, we can ensure that the probing set does not leak secret information.

4 A General Framework for First-Order Robust Probing Security

The security of a masked implementation fundamentally relies on compliance with the Unique Randomness Guard (URG) principle, as established in the preceding theoretical framework. This

section develops a systematic methodology for converting circuits into first-order probing-secure masked circuits.

The discussion is structured in two main parts. The first part introduces the general strategy for constructing a masked circuit under the assumption of sufficient fresh randomness. The second part focuses on the management of randomness in practical scenarios, where the availability of fresh random bits may be limited or constrained. In this context, the section presents techniques to maintain first-order probing security while optimizing the use of randomness, particularly in composable circuit settings.

4.1 Secure Compilation with Unbounded Randomness

4.1.1 The Naive Compiler and its Insecurity

The most direct approach to masking a circuit is the naive compiler. This strategy involves a one-to-one replacement of each standard gate with its corresponding secure gadget, under the assumption that an abundant supply of fresh randomness is available.

Linear Gates For linear gates such as XOR, the transformation is straightforward, as the operation can be computed share-wise without introducing leakage:

$$\begin{cases} y_0 = a_0 \oplus b_0 \\ y_1 = a_1 \oplus b_1 \end{cases}$$

Non-linear Gates Non-linear gates, particularly the AND gate, require fresh randomness to prevent information leakage. A secure AND gadget for a first-order scheme is implemented as:

$$\begin{cases} y_0 = [a_0 \otimes b_0 \oplus r_0] \oplus [a_0 \otimes b_1 \oplus r_1] \\ y_1 = [a_1 \otimes b_0 \oplus r_0] \oplus [a_1 \otimes b_1 \oplus r_1] \end{cases}$$

where r_0 and r_1 are fresh, independent random bits. The introduction of these fresh, independent random bits (r_0, r_1) is the key mechanism for upholding security. A primary strategy for satisfying the URG principle for the gadget's output is to leverage this newly introduced randomness to serve as the required unique random guards.

The Naive Compiler operates on the premise that the internal randomness introduced within each non-linear gadget will automatically secure the gadget's output, thereby ensuring the security of the entire circuit.

4.1.2 The Challenge: Guard Collisions

As noted in the previous section, the security of a combinational circuit follows once its inputs can be covered by a Unique Randomness Guard (URG) set. Hence, to establish the security of a masked implementation, we assign a randomness guard to each primary input and register output of the circuit.

Formally, for any wire w in the circuit, we denote by $\text{RandSet}(w)$ the set of randomness guards associated with the primary inputs and register outputs that serve as the combinational inputs leading to w . In other words, $\text{RandSet}(w)$ collects the designated guard of each input dependency of w , thereby providing a compact representation of the randomness structure on which w ultimately relies.

Within a single share, all registers and inputs receive different randomness elements; however, for the masked representation of a given sensitive value, the same randomness element necessarily appears across its shares to ensure correctness. Linear components, such as XOR gates, operate entirely within shares and do not introduce new registers; the guards of their outputs can naturally be taken as the union of the guards of their inputs, and such linear combinations do not introduce

additional leakage under the probing model. In contrast, non-linear components, such as AND gadgets, operate across shares, which can cause the same randomness element to appear in multiple inputs, leading to **guard collisions**.

To illustrate the occurrence of guard collisions, consider the masked computation

$$y = (a \oplus b) \otimes (a \oplus c),$$

where a , b , and c are sensitive variables with first-order masking shares (a_0, a_1) , (b_0, b_1) , and (c_0, c_1) , respectively. Expanding the shares, the inputs to the AND gadget become

$$(a_0 \oplus b_0, a_1 \oplus b_1) \quad \text{and} \quad (a_0 \oplus c_0, a_1 \oplus c_1).$$

Since both inputs involve shares of a , the randomness guard associated with a —used in constructing a_0 and a_1 —appears in both inputs of the AND gadget. Consequently, the internal fresh randomness of the AND gadget alone is insufficient to guarantee a unique guard for the output, potentially leading to a secret leakage.

Formally, we define a guard collision as follows. Let $\mathbf{x} = (x_0, x_1)$ and $\mathbf{y} = (y_0, y_1)$ be two sets of shares entering a non-linear gadget, and let $\text{RandSet}(\mathbf{x})$ and $\text{RandSet}(\mathbf{y})$ denote the sets of designated randomness guards associated with their combinational inputs. A *guard collision* occurs if

$$\text{RandSet}(\mathbf{x}) \cap \text{RandSet}(\mathbf{y}) \neq \emptyset.$$

This condition captures the precise scenario in which the same randomness element influences multiple inputs of a non-linear operation, thereby potentially compromising the uniqueness of the randomness guard for the output and violating the URG property.

4.1.3 The Secure Compiler Solution

To remedy the shortcomings of the naive compiler, a secure compiler is required. Its primary function is to systematically identify and eliminate guard collisions before they can compromise security. The central tool for this task is the refresh gadget. For masked inputs, the natural choice is the fresh randomness used in the masking itself. For register outputs, we likewise associate a designated random element introduced during the construction of the corresponding gadget—for instance, the fresh randomness inserted in the AND gadget is chosen as the guard of its output register.

The Refresh Gadget. The purpose of a refresh operation is to re-randomize a shared variable, breaking its existing random dependencies. For a shared variable $\mathbf{y} = (y_0, y_1)$, the refresh gadget introduces a fresh random bit r_{new} to produce a new set of shares $\mathbf{y}' = (y'_0, y'_1)$:

$$\begin{cases} y'_0 = [y_0 \oplus r_{\text{new}}] \\ y'_1 = [y_1 \oplus r_{\text{new}}] \end{cases}$$

The output $\mathbf{y}$ now has a new random dependency set $\text{guard}(\mathbf{y}') = \{r_{\text{new}}\}$, effectively severing its connection to any guards that previously protected $\mathbf{x}$.

Since the refresh operation introduces higher computational and randomness costs, the compiler applies it selectively. The algorithm, presented in Algorithm 1, intelligently deploys these tools to eliminate all guard collisions in the most resource-efficient way.

Security rationale. The correctness of the Cost-Aware Compiler is founded on a local invariant enforced at every non-linear gadget: before emitting a secure implementation of a non-linear operation the compiler ensures that the sets of designated randomness guards associated with the gadget’s combinational inputs permit the construction of a Unique Randomness Guard (URG) for the gadget output. For an AND gadget with inputs $\mathbf{a}$ and $\mathbf{b}$, the compiler inspects $\text{RandSet}(\mathbf{a})$ and $\text{RandSet}(\mathbf{b})$ and enforces one of three mutually exclusive corrective actions; the justification for each case is as follows.

Algorithm 1: A Cost-Aware Secure Compiler

Input : An unmasked circuit C ; A stream of fresh random bits.
Output A masked circuit $\hat{C}$.
 :
foreach *primary input* x **in** C **do**
 Add to $\hat{C}$: $x \leftarrow \text{Share}(x, \text{Rand.next}());$
foreach *gate* $g : z \leftarrow OP(a, b)$ **in topological order in** C **do**
 if g **is a linear gate** **then**
 Add to $\hat{C}$: $z \leftarrow \text{SecureOP}(a, b);$
 else if g **is an AND gate** **then**
 Let $\mathcal{R}_a$ and $\mathcal{R}_b$ be the random dependency sets for a and b ;
 if $\text{RandSet}(a) \cap \text{RandSet}(b) = \emptyset$ **then**
 Add to $\hat{C}$: $z \leftarrow \text{SecureAND}(a, b, \text{Rand.next}());$
 else if $\text{RandSet}(a) \setminus \text{RandSet}(b) \neq \emptyset$ **then**
 Add to $\hat{C}$: $z \leftarrow \text{SecureAND}([a], b, \text{Rand.next}());$
 else if $\text{RandSet}(b) \setminus \text{RandSet}(a) \neq \emptyset$ **then**
 Add to $\hat{C}$: $z \leftarrow \text{SecureAND}(a, [b], \text{Rand.next}());$
 else
 Add to $\hat{C}$: $z \leftarrow \text{SecureAND}(\text{Refresh}(a, \text{Rand.next}()), b, \text{Rand.next}());$
return $\hat{C}$

- **No collision** ($\text{RandSet}(a) \cap \text{RandSet}(b) = \emptyset$). If the two input guard sets are disjoint, then the gadget's internal randomness (the fresh random bits introduced by the SecureAND implementation) is independent of both inputs' guards. Hence the internal randomness together with the disjoint input guards yields a valid URG for the output: any single probe on the gadget's wires necessarily touches at least one randomness element from this URG and therefore reveals no secret information. In this case no additional transformation is required and the standard SecureAND can be applied directly.
- **Partial collision** ($\text{RandSet}(a) \cap \text{RandSet}(b) \neq \emptyset$, **but** $\text{RandSet}(a) \setminus \text{RandSet}(b) \neq \emptyset$ **or** $\text{RandSet}(b) \setminus \text{RandSet}(a) \neq \emptyset$). In this case at least one input carries a guard that is not present on the other input; such a unique guard can be used (together with the gadget's internal randomness) to form a URG for the output. To make this usable at the level of combinational inputs the compiler may insert a register on an appropriate path. Register insertion is a logical transformation that changes the combinational input boundary seen by the gadget (i.e., it makes the gadget observe the register output as the input), and thus it can be used to materialize the unique guard at the gadget input without altering the underlying randomness pool. Put differently, inserting a register redefines which register outputs (and hence which designated guards) serve as the combinational inputs; doing so allows the compiler to satisfy the URG precondition using existing, uniquely occurring guards while avoiding the cost of a full refresh.
- **Severe collision** ($\text{RandSet}(a) = \text{RandSet}(b)$). If one input's guard set is entirely contained in the other's, there is no guard that is algebraically unique to one input. In this situation the gadget's internal randomness alone cannot serve as a URG because the overlapping guard(s) propagate into both operands and remain present after the non-linear combination. The compiler therefore must perform a Refresh on (at least) one input: the Refresh operation algebraically rewrites the input shares by consuming a fresh random bit and thereby replaces the original guards with a new, independent guard. After refresh the input's RandSet is updated to include the new guard and excludes the previous conflicting ones; at that point

the URG precondition can be satisfied and the SecureAND can be applied.

By construction the compiler enforces the following loop invariant while processing gates in topological order: when emitting the secure implementation of any non-linear gadget, the guard sets of its combinational inputs are such that either (i) they are disjoint, or (ii) at least one input contains a guard absent from the other (and this uniqueness is made manifest at the combinational boundary by register insertion), or (iii) a Refresh has been applied so that an independent guard is available. This invariant guarantees that every non-linear gadget satisfies the URG precondition at the time of its instantiation. Because the transformation is local and the invariant is preserved gate by gate, the final compiled circuit satisfies the URG property at every combinational point; consequently, it is secure in the first-order probing model.

Costs and heuristics. In practice the compiler applies inexpensive measures (register insertion) whenever they suffice, and resorts to Refresh only when necessary; this yields a cost-aware tradeoff between randomness consumption and circuit overhead while preserving the security invariant above.

4.2 Randomness Reusing in Composable Circuits

In practical hardware implementations, the supply of fresh randomness is often a limited and costly resource. Consequently, developing techniques for reducing randomness consumption without compromising security is essential. The security of such optimizations hinges on maintaining the desired security properties, such as the Unique Randomness Guards (URG), at the boundaries of interconnected circuit blocks.

The Challenge of Randomness Dependency. Let us consider a masked circuit C that takes a secret input $\mathcal{X}$ and a set of random numbers $\mathcal{R}$ to produce a masked output $\mathbf{y} = (y_0, y_1)$. The output shares can be expressed as complex Boolean functions of the inputs and the randomness: $y_i = F_{\mathcal{X} \cup \mathcal{R}}^i$. A fundamental property of a correct masking scheme is that the unmasked, true value must be independent of the randomness, i.e., $y_0 \oplus y_1 = F_{\mathcal{X}}$. However, each individual share, y_0 and y_1 , retains a complex algebraic dependency on the entire set of random variables $\mathcal{R}$ used within the circuit. If we were to reuse any random variable from $\mathcal{R}$ in a subsequent computation that also involves y_0 or y_1 , we would create unintended correlations, potentially leading to security flaws. This dependency makes the safe reuse of randomness a non-trivial challenge.

Randomness Recycling Gadget To enable the safe recycling of the random numbers in $\mathcal{R}$, we must first sever the dependency between the output shares and $\mathcal{R}$. This is achieved via an output *re-masking* operation using a single fresh random bit r' , which is by definition independent of $\mathcal{R}$. The operation is as follows:

$$\begin{cases} y'_0 = [[y_0 \oplus r'] \oplus y_1] = F_{\mathcal{X}}^y \oplus r' \\ y'_1 = r' \end{cases}$$

This transformation elegantly preserves the encoded value ($y'_0 \oplus y'_1 = F_{\mathcal{X}}^y$) while ensuring that the new shares (y'_0, y'_1) now depend *only* on the fresh random bit r' . The original, complex dependency on $\mathcal{R}$ is completely eliminated.

As a result, the value y'_0 is now statistically independent of $\mathcal{R}$, allowing it to be securely processed in subsequent steps that might reuse randomness from $\mathcal{R}$.

Rules for Composition. The principle of URG properties underpins the secure reusing of randomness when composing smaller subcircuits into a larger design.

- **Parallel Composition.** Consider two masking subcircuits, $\hat{C}_A$ and $\hat{C}_B$, operating in parallel. If their initial input masking sets, $\mathcal{R}_{in}^{(A)}$ and $\mathcal{R}_{in}^{(B)}$, are disjoint ($\mathcal{R}_{in}^{(A)} \cap \mathcal{R}_{in}^{(B)} = \emptyset$), then from the perspective of $\hat{C}_A$, any random variable in $\mathcal{R}_{in}^{(B)}$ is indistinguishable from a fresh random source. Consequently, it is secure for $\hat{C}_A$ to "borrow" an input random guard from $\hat{C}_B$ to use as an internal random bit, enabling a form of randomness sharing.
- **Serial Composition.** When subcircuits are composed serially, $\hat{C}_1 \rightarrow \hat{C}_2$, the security of $\hat{C}_2$ is contingent upon its inputs possessing a valid URG property. Specifically, each input share fed to $\hat{C}_2$ must be masked by an independent and unique random variable. Since the computations within $\hat{C}_1$ may introduce complex algebraic dependencies among its output shares (as discussed above), it is often necessary to insert a re-masking or a refresh gadget at the interface. This gadget's role is to re-establish the URG property for $\hat{C}_2$'s inputs, thus ensuring the composite circuit remains secure.

In summary, the management of randomness in a composable circuit framework follows a systematic logic. In parallel compositions, disjointness of the input random sets allows for safe sharing of randomness across subcircuits without introducing correlations. For serial compositions or scenarios requiring randomness recycling, re-masking plays a critical role: it decouples the output shares from previously used random variables, effectively restoring the independence and uniqueness of the random masks at the interface of downstream components. Consequently, operations such as re-masking and careful management of randomness are essential for preserving the independence and uniqueness of random masks at subcircuit boundaries, ensuring that downstream subcircuits maintain the URG property and preventing unintended correlations that could compromise security.

5 Case Design on first-order secure AES masking with only randomness for input encoding

In this section, we instantiate our theoretical framework by applying the proposed secure compilation methodology to a standard cryptographic primitive: the Advanced Encryption Standard (AES). The objective is to demonstrate how the principles of unique randomness guards preservation and composable randomness management, introduced in Section 3, translate into a practical, low-randomness masked implementation.

We will first detail the fully parallel, round-based hardware architecture of our design. Then, we will explain how our compiler is applied to the 20 concurrently operating masked S-boxes. Subsequently, we will elaborate on the core innovation of this implementation: a randomness recycling strategy, rigorously justified by our rules for parallel and serial composition. Finally, we present a comprehensive security analysis, including a formal proof argument, automated verification results, and an experimental evaluation using Test Vector Leakage Assessment (TVLA).

5.1 AES Design

This section presents the design of a masked AES round function that preserves the URG property throughout its computation. Our construction begins with the presupposition that the input state to the round function already possesses a well-defined URG structure. The core challenge is to demonstrate that this property is maintained through the composition of the two distinct transformations within the round: the linear layer and the non-linear layer. Furthermore, the output of each layer is stored into registers.

The preservation of the URG property through the linear layer is a direct consequence of theorem 2. Consequently, a direct, share-wise implementation of the linear layer is provably secure. Addressing the non-linear layer, which consists of multiple S-boxes operating in parallel, demands

a more sophisticated approach. To circumvent the high cost of introducing fresh randomness for each S-box, our strategy is to leverage the randomness across the parallel inputs. Each S-box gadget is designed to source its internal randomness from the inputs of adjacent S-boxes. However, the non-linear nature of the S-box transformation disrupts the simple URG structure at the output. To restore URG property, the output of each S-box is re-masked by cross-using inputs from the other parallel S-boxes as a fresh mask.

5.1.1 Overall Hardware Architecture

Our design implements a round-based, parallel hardware architecture for the AES algorithm, engineered to provide first-order security in the probing model. The architecture is logically partitioned into two primary datapaths that operate concurrently: the Encryption Datapath and the on-the-fly Key-Expansion Datapath. A central feature is the strategic management of masked data and the internal sourcing of randomness for non-linear operations.

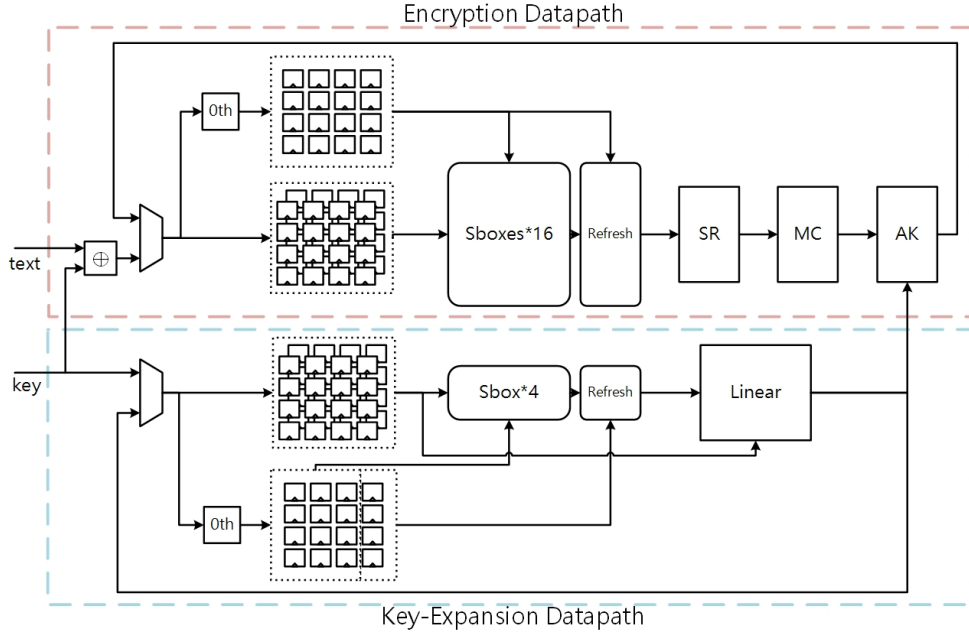


Figure 1: Dataflow diagram: Masked AES hardware architecture

Encryption Datapath. As shown in Fig. 1, the encryption datapath is organized around a set of State Registers that hold the intermediate encryption state. At initialization, these registers are loaded with the masked plaintext and the corresponding key input. In the first round, a bank of input MUXes selects between the initial value and the feedback path, establishing a uniform datapath structure; in subsequent rounds, the state is always derived from the output of the `ADDROUNDKEY` (AK) stage of the previous round. The selected state is processed by sixteen parallel masked S-boxes (SB), followed by a re-masking layer. Randomness for these non-linear operations is implicitly derived from the 0^{th} share registers, which are maintained separately from the main State Registers; the precise allocation policy is detailed in section 5.1.2. The following linear mappings—`SHIFTROWS` (SR) and `MIXCOLUMNS` (MC)—are applied per share. The transformed state is then combined with the appropriate round key in the AK stage, and the result is written back to the State Registers.

Key-Expansion Datapath. Round keys are generated on-the-fly by a dedicated key-expansion datapath composed of round key registers for all shares. At initialization, the external cipher key is loaded through a MUX that temporarily replaces the feedback path used in later rounds; after the first update, the key words are always derived from the output of the previous round key. Each update cycle applies the non-linear SUBWORD (SW) transformation—implemented with masked S-boxes—to the final word of the previous round key. In keeping with the shared-resource policy, randomness for these masked S-boxes is drawn from the 0^{th} shares of the remaining key words, which are stored in dedicated registers separate from the main Round Key Registers, while output refreshing uses distinct bytes of the same 0^{th} share within the processed word. The subsequent linear operations—ROTWORD (RW) and round-constant addition (RC)—are applied share-wise. The resulting round key is stored for the next iteration and concurrently supplied to the AK stage of the encryption datapath.

5.1.2 Implementing the Non-Linear Layer

The masked S-box is generated by processing the Boolean circuit representation from Listing 16 in [JBKK25] with our compiler. This specific circuit structure, which has a multiplicative depth of 4, consists of 34 AND gates and 81 linear gates. Consequently, the resulting masked S-box implementation requires 68 random bits for the AND gates and an additional 8 random bits for its output refreshing operation.

A core feature of this architecture is its ability to operate without a dedicated randomness generator by sourcing the required randomness for each S-box from the input shares of its concurrently operating peers. To formalize this allocation for the 16 parallel S-boxes, let $S_{i,j}^k$ denote the k -th share of the input to the S-box at row i and column j . For the S-box at position (i, j) , the 8 bits for its output refreshing are taken from the input share $S_{i,j+1+(i \bmod 2)}^0$. The 68 bits for its internal AND gates are then sourced from the input shares of the remaining 14 S-boxes. As illustrated in Fig. 2, this deterministic allocation scheme ensures that the Randomness Guards protecting the outputs of the parallel S-boxes are mutually disjoint, thereby upholding the URG property for the entire state.



Figure 2: Randomness allocation

5.2 Security and performance Analysis

5.2.1 Formal Verification of the Masked S-box

To formally assess the probing security of our design, we focus our analysis on the S-box, as it is the sole non-linear component and thus the most critical element for ensuring the security of the entire implementation. We use the publicly available Coco framework [HB21] to verify the security of our S-box implementation within the time-constrained probing model.

The verification process is configured as follows. First, our design is synthesized into a gate-level netlist using Yosys [20225]. We then label the circuit’s primary inputs for Coco. The

masked input to the S-box is marked as sensitive. The 74 bits of randomness required are declared as fresh and uniformly random for each clock cycle of the S-box's operation.

After running the analysis, Coco confirmed the first-order security of our S-box implementation. This result provides strong evidence that our S-box gadget is secure against first-order probing attacks, forming a solid foundation for the security of the full cipher round.

5.2.2 Security Proof

This section provides a formal proof for the proposed AES hardware implementation under the first-order robust probing model, based on the security framework established in previous sections.

The security of the implementation is argued by analyzing its constituent layers and their composition. We first establish the security of the non-linear layer. This layer, consisting of parallel S-boxes, is secured by a randomness allocation strategy that strictly adheres to the rule for parallel composition. As depicted in Fig. 2, this strategy is critical as it ensures that the collective output of the entire non-linear layer satisfies the URG property.

With the security of the non-linear layer established, we analyze the subsequent linear operations. Since linear operations (SR, MC, AK, RW, RC) are computed share-wise and are thus inherently secure in the first-order model. All linear transformations are bijective. Therefore, as established by the preceding theorem 2, they preserve the URG property of the data they process. Consequently, the entire round's output preserves the URG property established by the non-linear layer, satisfying the rule for serial composition.

Since all components and their compositions strictly follow the established security rules for both parallel and serial composition, we conclude that the design is secure in the first-order probing model.

5.2.3 Experimental Evaluation

To practically verify the effectiveness of our implemented scheme against side-channel attacks, we conducted an experimental evaluation using the TVLA framework [GJR11]. TVLA is a widely adopted, statistics-based methodology that formally tests for distinguishable differences in a device's power consumption when processing different classes of input data. The framework's null hypothesis (H_0) posits that no such statistical difference exists. A resulting Welch's t-statistic that remains within the established confidence threshold of ± 4.5 indicates that no significant side-channel leakage is detected. In this case, the null hypothesis is not rejected, and the design definitively passes the TVLA test.

Experimental Platform Our crypto-implementation was deployed on a NewAE CW305 evaluation board, which features a Xilinx Artix-7 FPGA. We captured power traces using a PicoScope oscilloscope over a shunt resistor on the FPGA's power line. To mitigate measurement noise, we adopted the same techniques as in [GKM⁺24], namely synchronizing the clocks between the FPGA and the oscilloscope and applying a pre-processing step to ensure precise alignment of the traces. It is noteworthy that the evaluation board used does not integrate any dedicated hardware Random Number Generator instances.

Evaluation Methodology Our specific testing methodology was the fixed-versus-random t-test. We created two distinct sets of power traces: a "fixed set" by repeatedly encrypting a constant plaintext, and a "random set" by encrypting a new random plaintext for each trace. A constant encryption key was used throughout all measurements. To eliminate time-varying environmental factors, the encryptions for both sets were performed in a randomly interleaved manner. This evaluation proceeded in two distinct phases. First, to confirm the soundness of our measurement setup, we analyzed the implementation with its masking RNG disabled, for which we collected 10^4 traces. Subsequently, we evaluated the definitive configuration with the RNG enabled, using 10^7 traces.

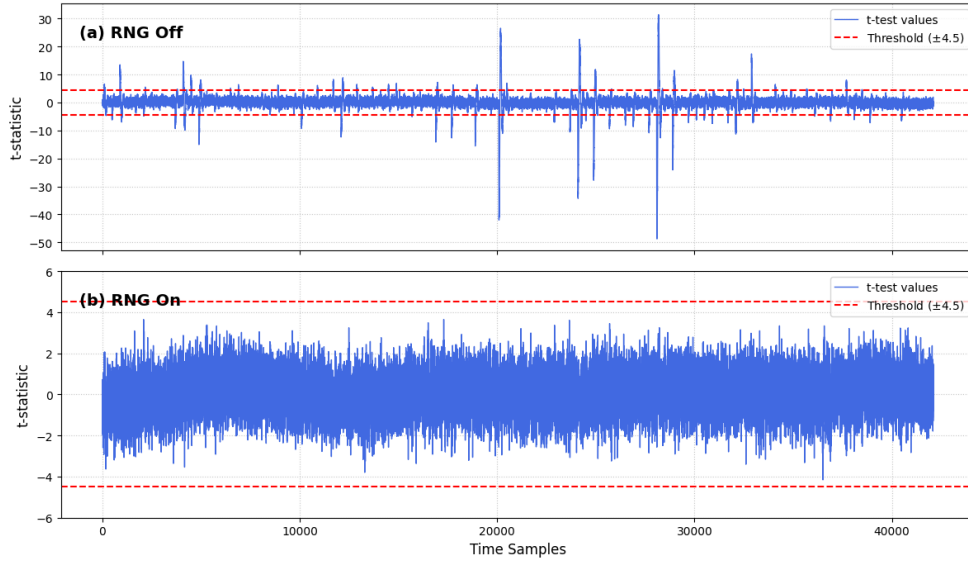


Figure 3: TVLA t-test results with the masking RNG disabled (top) and enabled (bottom).

Results Our experimental results clearly demonstrate the effectiveness of the masking scheme. In the control experiment with the RNG disabled, the design exhibited first-order leakage as expected (results not shown here for brevity). However, as shown in Figure 3, in our main experiment with the RNG enabled, the t-statistic never exceeded the significance threshold of ± 4.5 , even with up to 10^7 traces. This result provides strong experimental evidence that our implementation can effectively resist first-order DPA attacks.

5.3 Implementation Results and Comparison

To evaluate the hardware area overhead of our proposed AES implementation, we first utilize the sv2v [Sno25] tool to convert the SystemVerilog implementation into synthesizable Verilog code. Subsequently, we employ the open-source synthesis tool Yosys [20225] and synthesize the design based on the NanGate 45nm [The] open-source standard cell library to measure its gate area. The reported S-box area includes its fundamental logic as well as the required refreshment circuitry.

Our performance comparison primarily focuses on AES implementations with low randomness consumption. For round-based implementations, we will conduct a comprehensive analysis of both area cost and the efficiency of randomness usage. For serial architectures, our comparison will primarily emphasize the efficiency of randomness usage, as these designs are inherently optimized for minimal area, making a direct area comparison less meaningful.

As shown in Table 1, our implementation is the first hardware implementation to achieve zero fixed randomness consumption. All prior state-of-the-art hardware implementations either rely on a static pool of tens to hundreds of random bits, pre-allocated to satisfy various internal requirements of the masking scheme [WM18, Sug18, ADN⁺21], or necessitate online randomness generation [YCW⁺24]. Additionally, a prior proposal introduces the use of 2-bit randomness (including randomness for input encoding); however, this approach has only been validated in software, and its theoretical hardware implementation is estimated to incur a latency of 24 clock cycles per S-box, which is impractical for real-world applications [GSDM⁺19].

In contrast, our design substantiates the feasibility of implementing first-order secure AES masking using solely the initial randomness required for plaintext and key sharing, while maintaining a competitive area footprint and acceptable latency. In terms of performance, our implementation incurs an area overhead of 66.87 kGE, which is comparable to the most compact

Table 1: Comparison of hardware implementation overheads for different first-order side-channel countermeasures

Design	Area (kGE)		Latency (cycles)		S-box share	Randomness (bits)		
	S-box	AES	S-box	AES		S-box	AES	Fixed*
[WM18]	–	–	16	–	4	0	0	384
[Sug18]	–	–	4	–	3	0	0	264
[ADN ⁺ 21] Design1	2.71	63.91	5	62	2	0	0	54
[ADN ⁺ 21] Design2	4.33	102.44	4	52	2	0	0	36
[ADN ⁺ 21] Design3	5.84	115.73	3	42	2	0	0	40
[YCW ⁺ 24] Design1	–	–	6	–	2	4	800	0
[YCW ⁺ 24] Design2	–	–	6	–	2	8	1600	0
This Work	2.70	66.87	9	101	2	0	0	0

* “Fixed Randomness” denotes all other initial random numbers required by the design, excluding those used for the initial masking of the plaintext and key.

designs reported in the literature for similar schemes. It should be emphasized that this work serves as an initial validation of our core methodology. Further architectural optimizations are expected to enhance latency performance and overall efficiency.

6 Conclusion and Future Work

A central challenge in hardware masking is managing its high randomness cost while maintaining provable security. While randomness reuse is key to efficiency, the resulting algebraic dependencies complicate formal, multi-round security proofs. To date, ensuring provable security under such conditions has consistently required the introduction of extra randomness, either from online RNGs or as additional initial states. Consequently, a provably secure masking scheme that operates with zero extra randomness had not been achieved.

This paper resolves this long-standing challenge by introducing a novel theoretical framework centered on the Unique Randomness Guard (URG) property. Grounded on this cornerstone property, we derive a systematic methodology for randomness reuse, embodied in a set of formal rules for parallel and serial composition. This framework provides a clear path to manage and prove the security of the complex dependencies introduced by reuse, thereby enabling us to completely eliminate the reliance on any extra randomness.

We instantiated our URG-based framework in a first-order secure AES hardware implementation. This design is the first to achieve provable security with zero extra randomness beyond the initial masking of the plaintext and key, all while maintaining a competitive hardware area (66.87 kGE). Its security was comprehensively validated through formal proofs, automated verification using the Coco tool, and extensive TVLA test on an FPGA platform, where no first-order leakage was detected after collecting 10^7 power traces.

This work proves that efficient randomness reuse and rigorous provable security can be achieved simultaneously, opening up a versatile new approach for designing secure hardware in resource-constrained environments. This work lays a foundation for future research in two key directions. The presented AES implementation, while validating our methodology with competitive area, can be further architecturally optimized to reduce its latency. More broadly, the URG framework itself can now be applied to other cryptographic primitives, especially lightweight ciphers like PRESENT or SKINNY, where its principles can be leveraged to address critical randomness constraints.

References

- [20225] YosysHQ/yosys. Yosys Headquarters, October 2025.
- [ADN⁺21] Amund Askeland, Siemen Dhooghe, Svetla Nikova, Vincent Rijmen, and Zhenda Zhang. Guarding the First Order: The Rise of AES Maskings, 2021.
- [BDRS22] Tim Beyne, Siemen Dhooghe, Adrián Ranea, and Danilo Sijacic. A low-randomness second-order masked AES. In Riham AlTawy and Andreas Hülsing, editors, *SAC 2021*, volume 13203 of *LNCS*, pages 87–110. Springer, Cham, September / October 2022.
- [BGN⁺15] Begül Bilgin, Benedikt Gierlichs, Svetla Nikova, Ventzislav Nikov, and Vincent Rijmen. Trade-Offs for Threshold Implementations Illustrated on AES. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 34(7):1188–1200, July 2015.
- [CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 398–412. Springer, Berlin, Heidelberg, August 1999.
- [CPRR14] Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Thomas Roche. Higher-order side channel security and mask refreshing. In Shiho Moriai, editor, *FSE 2013*, volume 8424 of *LNCS*, pages 410–424. Springer, Berlin, Heidelberg, March 2014.
- [DBR19] Lauren De Meyer, Begül Bilgin, and Oscar Reparaz. Consolidating security notions in hardware masking. *IACR TCHES*, 2019(3):119–147, 2019.
- [DNR19] Siemen Dhooghe, Svetla Nikova, and Vincent Rijmen. Threshold implementations in the robust probing model. Cryptology ePrint Archive, Report 2019/1005, 2019.
- [DRB⁺16] Thomas De Cnudde, Oscar Reparaz, Begül Bilgin, Svetla Nikova, Ventzislav Nikov, and Vincent Rijmen. Masking AES with $d+1$ shares in hardware. In Benedikt Gierlichs and Axel Y. Poschmann, editors, *CHES 2016*, volume 9813 of *LNCS*, pages 194–212. Springer, Berlin, Heidelberg, August 2016.
- [FGM⁺18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and François-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR TCHES*, 2018(3):89–120, 2018.
- [GJJR11] G. Goodwill, Benjamin Jun, J. Jaffe, and P. Rohatgi. A testing methodology for side channel resistance validation. 2011.
- [GKM⁺24] Barbara Gigerl, Franz Klug, Stefan Mangard, Florian Mendel, and Robert Primas. Smooth passage with the guards: Second-order hardware masking of the AES with low randomness and low latency. *IACR TCHES*, 2024(1):309–335, 2024.
- [GMK17] Hannes Groß, Stefan Mangard, and Thomas Korak. An efficient side-channel protected AES implementation with arbitrary protection order. In Helena Handschuh, editor, *CT-RSA 2017*, volume 10159 of *LNCS*, pages 95–112. Springer, Cham, February 2017.
- [GP99] Louis Goubin and Jacques Patarin. DES and differential power analysis (the “duplication” method). In Çetin Kaya Koç and Christof Paar, editors, *CHES'99*, volume 1717 of *LNCS*, pages 158–172. Springer, Berlin, Heidelberg, August 1999.

- [GSDM⁺19] Hannes Gross, Ko Stoffelen, Lauren De Meyer, Martin Krenn, and Stefan Mangard. First-Order Masking with Only Two Random Bits. In *Proceedings of ACM Workshop on Theory of Implementation Security Workshop, TIS'19*, pages 10–23, New York, NY, USA, November 2019. Association for Computing Machinery.
- [HB21] Vedad Hadžić and Roderick Bloem. COCOALMA: A Versatile Masking Verifier. In *2021 Formal Methods in Computer Aided Design (FMCAD)*, pages 14–23, October 2021.
- [ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 463–481. Springer, Berlin, Heidelberg, August 2003.
- [JBKK25] Yongjin Jeon, Seungjun Baek, Giyoon Kim, and Jongsung Kim. A Framework for Generating S-Box Circuits with Boyar–Peralta Algorithm-Based Heuristics, and Its Applications to AES, SNOW3G, and Saturnin. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):586–631, 2025.
- [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 388–397. Springer, Berlin, Heidelberg, August 1999.
- [Koc96] Paul C. Kocher. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In Neal Koblitz, editor, *CRYPTO'96*, volume 1109 of *LNCS*, pages 104–113. Springer, Berlin, Heidelberg, August 1996.
- [Mes01] Thomas S. Messerges. Securing the AES finalists against power analysis attacks. In Bruce Schneier, editor, *FSE 2000*, volume 1978 of *LNCS*, pages 150–164. Springer, Berlin, Heidelberg, April 2001.
- [MPL⁺11] Amir Moradi, Axel Poschmann, San Ling, Christof Paar, and Huaxiong Wang. Pushing the limits: A very compact and a threshold implementation of AES. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 69–88. Springer, Berlin, Heidelberg, May 2011.
- [NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *ICICS 06*, volume 4307 of *LNCS*, pages 529–545. Springer, Berlin, Heidelberg, December 2006.
- [RBN⁺15] Oscar Reparaz, Begül Bilgin, Svetla Nikova, Benedikt Gierlichs, and Ingrid Verbauwhede. Consolidating masking schemes. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 764–783. Springer, Berlin, Heidelberg, August 2015.
- [RP10] Matthieu Rivain and Emmanuel Prouff. Provably secure higher-order masking of AES. In Stefan Mangard and François-Xavier Standaert, editors, *CHES 2010*, volume 6225 of *LNCS*, pages 413–427. Springer, Berlin, Heidelberg, August 2010.
- [SM21] Aein Rezaei Shahmirzadi and Amir Moradi. Re-consolidating first-order masking schemes. *IACR TCHES*, 2021(1):305–342, 2021.
- [Sno25] Zachary Snow. Sv2v: SystemVerilog to Verilog, October 2025.
- [Sug18] Takeshi Sugawara. 3-share threshold implementation of AES s-box without fresh randomness. *IACR TCHES*, 2019(1):123–145, 2018.

- [The] The-OpenROAD-Project. NANGATE: The NanGate 45nm Open Cell Library. <https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts/tree/master/flow/platforms/nangate45>.
- [UHA17] Rei Ueno, Naofumi Homma, and Takafumi Aoki. Toward more efficient DPA-resistant AES hardware architecture based on threshold implementation. In Sylvain Guilley, editor, *COSADE 2017*, volume 10348 of *LNCS*, pages 50–64. Springer, Cham, April 2017.
- [WM18] Felix Wegener and Amir Moradi. A first-order SCA resistant AES without fresh randomness. In Junfeng Fan and Benedikt Gierlichs, editors, *COSADE 2018*, volume 10815 of *LNCS*, pages 245–262. Springer, Cham, April 2018.
- [YCW⁺24] Fu Yao, Hua Chen, Yongzhuang Wei, Enes Pasalic, Feng Zhou, and Limin Fan. Optimizing AES Threshold Implementation Under the Glitch-Extended Probing Model. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(7):1984–1997, July 2024.